

CY Cergy-Paris Université

RAPPORT

pour le projet Génie Logiciel du Projet (GLP)
Licence 2 Informatique

sur le sujet

CHESS
Réalisation du jeu d'échecs chinois

rédigé par

DJERMOUNE Thanina - KETFI Ramdane - SOW Mamadou Saliou

Avril 2026

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectif du projet	3
1.3	Organisation du rapport	3
2	Spécification du projet	4
2.1	Notions de base et contraintes du projet	4
2.1.1	Fonctionnement général du logiciel	4
2.1.2	Notions et terminologies de base	4
2.1.3	Contraintes et limitations connues	4
2.2	Fonctionnalités attendues du projet	5
3	Conception et mise en œuvre	6
3.1	Architecture globale du logiciel	6
3.2	Packages	6
3.3	Classes de données	7
3.3.1	La classe Plateau	7
3.3.2	Initialisation du plateau	7
3.3.3	Méthodes principales	7
3.3.4	La hiérarchie des pièces	8
3.4	IHM Graphique	8
3.5	Conception des traitements (processus)	9
3.5.1	Diagramme de sequence	9
3.5.2	Les niveaux du bot	9
3.5.3	Validation des coups	10
3.5.4	Tests unitaires	10
4	Manuel utilisateur	11
4.1	Prérequis	11
4.2	Importer et configurer le projet sous Eclipse	11
4.3	Utilisation du jeu	11
5	Déroulement du projet	13
5.1	Réalisation du projet par étapes	13
5.2	Répartition des tâches, communication et intégration	13
6	Conclusion et perspectives	14
6.1	Résumé du travail réalisé	14
6.2	Améliorations possibles du projet	14

Liste des tableaux

1	Les pièces de chaque joueur	4
2	Valeurs des pièces utilisées par le bot	10

Remerciements

Les auteurs du projet tiennent à exprimer leur sincère gratitude à leur enseignant, Tianxiao LIU, pour son encadrement, sa disponibilité et ses précieux conseils tout au long de ce projet de Génie logiciel.

Ils souhaitent également remercier l'ensemble des enseignants et intervenants ayant contribué à leur formation, ainsi que leur établissement pour les moyens mis à leur disposition.

Enfin, ils adressent leurs remerciements à leurs camarades et à leurs proches pour leur soutien et leur collaboration.

1 Introduction

1.1 Contexte du projet

Dans le cadre de notre cours de Génie Logiciel du Projet en Licence 2 Informatique à CY Cergy-Paris Université, on nous a demandé de concevoir et développer un jeu en Java. Notre groupe a choisi de réaliser une version modifiée du jeu d'échecs chinois, qu'on a appelé **CHESS**. Ce choix vient de notre intérêt commun pour les jeux de stratégie, et ça nous permettait de travailler sur quelque chose de concret tout en appliquant ce qu'on a vu en cours, comme la séparation entre le moteur de jeu et l'interface graphique.

Les éléments présentés dans ce rapport correspondent à l'ensemble du travail réalisé par notre groupe dans le cadre du projet Génie Logiciel.

1.2 Objectif du projet

L'objectif était de faire un jeu jouable avec une interface graphique, un moteur qui gère toutes les règles correctement, et un bot pour jouer contre l'ordinateur. On voulait que le jeu soit complet et fonctionnel, pas juste une démonstration.

1.3 Organisation du rapport

Dans ce rapport, on présente les sections suivantes :

1. Les spécifications du projet.
2. La conception et le développement du logiciel.
3. Le manuel utilisateur.
4. Le déroulement du projet en équipe.
5. La conclusion et les perspectives d'amélioration.

2 Spécification du projet

2.1 Notions de base et contraintes du projet

2.1.1 Fonctionnement général du logiciel

CHESS est un jeu de stratégie à deux joueurs qui se jouent chacun à leur tour. Chaque joueur contrôle un ensemble de pièces et le but est de capturer le Général adverse. La partie se termine quand un joueur est en échec et mat, c'est-à-dire qu'il n'a plus aucun coup légal pour protéger son Général.

2.1.2 Notions et terminologies de base

Le plateau Le plateau de jeu fait **11 × 11 cases** et contient plusieurs zones :

- 9 colonnes jouables × 10 lignes jouables où les pièces se déplacent normalement ;
- le **palais**, une zone de 3×3 cases au centre de chaque côté du plateau, dans laquelle le Général et les Conseillères sont obligés de rester ;
- **2 colonnes neutres** (colonnes 3 et 7) qui agissent comme des murs — aucune pièce ne peut s'y arrêter, et elles ne comptent pas dans les distances de déplacement ;
- **1 ligne de rivière** (ligne 5) qui sépare les deux camps et affecte les déplacements des Soldats et des Éléphants.

Chaque joueur commence avec **16 pièces** réparties comme suit : Comme ce qui est illustré dans le

Pièce	Qté	Rôle
Général	1	Pièce principale, sa capture = fin de partie
Conseillère	2	Protège le Général dans le palais
Éléphant	2	Défensif, ne traverse pas la rivière
Chevalier	2	Se déplace en L
Chariot	2	Pièce la plus puissante en déplacement
Canon	2	Capture uniquement par-dessus un écran
Soldat	5	Avance d'une case, plus mobile après la rivière

TABLE 1 – Les pièces de chaque joueur

tableau 1, les deux joueurs ont exactement les mêmes pièces au départ.

2.1.3 Contraintes et limitations connues

Règles strictes Certaines règles sont strictement imposées par le moteur de jeu :

- Un joueur ne peut jamais terminer son tour avec son Général en position d'être capturé (interdit de se mettre en échec) ;
- Les deux Généraux ne peuvent jamais se retrouver sur la même colonne sans qu'une pièce les sépare — c'est la règle du **face à face** ;
- Tout coup qui ne respecte pas les règles est automatiquement refusé, le joueur doit rejouer.

Règles de déplacement Les règles de déplacement de chaque pièce sont les suivantes :

Le Général Se déplace d'une seule case horizontalement ou verticalement, uniquement dans le palais. Il ne peut jamais aller sur une case menacée par l'adversaire.

Les Conseillères Se déplacent d'une seule case en diagonale, uniquement dans le palais. Leur rôle principal est de protéger le Général.

Les Éléphants Se déplacent de deux cases en diagonale. Si la case intermédiaire est occupée, le déplacement est bloqué. Ils ne peuvent pas traverser la rivière, donc ils restent toujours dans leur moitié du plateau.

Les Chariots Se déplacent d'autant de cases qu'ils veulent horizontalement ou verticalement, à condition que le chemin soit complètement libre.

Les Canons Sans capture, ils bougent exactement comme les Chariots. Pour capturer, il faut qu'il y ait exactement une pièce (amie ou adverse) entre le Canon et sa cible — on appelle ça l'écran.

Les Soldats Avant de traverser la rivière, ils avancent uniquement d'une case vers l'avant. Après la rivière, ils peuvent aussi se déplacer latéralement. Ils ne reculent jamais.

Outils de développement utilisés dans ce projet :

1. Langage de programmation : Java (JRE 1.8)
2. IDE : Eclipse
3. Outil de rédaction du rapport : $\text{\LaTeX}$
4. Bibliothèque de logs : Log4j 1.2.17
5. Tests unitaires : JUnit

2.2 Fonctionnalités attendues du projet

Fonctionnalités Ce que l'utilisateur peut faire avec CHESS :

- Voir le plateau avec les colonnes neutres affichées différemment des cases normales ;
- Cliquer sur une pièce pour la sélectionner et voir directement les cases où elle peut aller ;
- Jouer contre l'ordinateur (**VS BOT**) avec trois niveaux : Facile, Moyen, Difficile ;
- Jouer à deux sur la même machine (**VS HUMAIN**) ;
- Consulter l'historique des coups joués pendant la partie ;
- Voir les pièces capturées de chaque côté ;
- Être averti automatiquement en cas d'échec ou d'échec et mat.

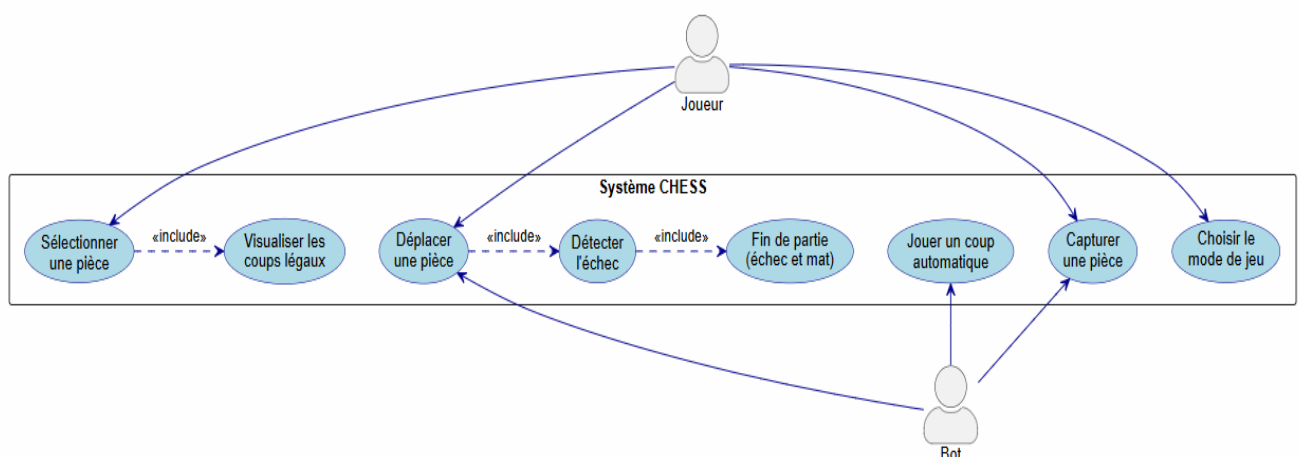


FIGURE 1 – Diagramme de cas d'utilisation

3 Conception et mise en œuvre

3.1 Architecture globale du logiciel

On a choisi une architecture MVC (**Modèle-Vue-Contrôleur**) pour bien séparer le moteur de jeu de l'interface graphique. Cette séparation nous a permis de travailler plus facilement en équipe et de tester chaque partie indépendamment.

- **Modèle** : les packages `engine.map` et `engine.mobile` gèrent le plateau et les pièces ;
- **Contrôleur** : le package `engine.process` gère la logique du jeu et le bot ;
- **Vue** : le package `gui` s'occupe de tout l'affichage avec Java Swing.

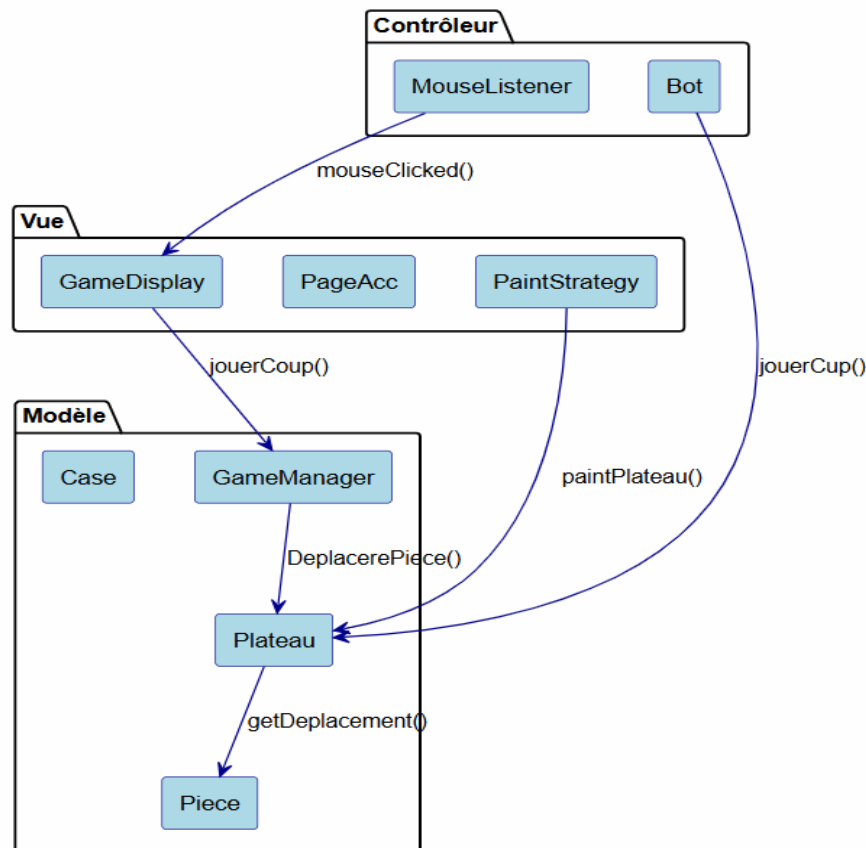


FIGURE 2 – Architecture MVC du projet

3.2 Packages

Voici la structure complète des packages du projet :

- `config` : constantes globales comme la taille du plateau et des cases ;
- `engine.map` : contient `Plateau`, `Case`, `CaseNormal`, `CaseMort`, `Couleur`, `TypeCaseNormale` ;
- `engine.mobile` : contient la classe abstraite `Piece` et ses sous-classes `General`, `Conseillere`, `Elephant`, `Chevalier`, `Char`, `Canon`, `Soldat` ;
- `engine.process` : contient `GameManager`, `Bot`, `Coup`, `Niveau` ;
- `gui` : contient `MainGUI`, `PageAcc`, `GameDisplay`, `PaintStrategy` ;
- `log` : configuration des logs avec `Log4j` ;
- `junit` : tests unitaires avec `JUnit`.

3.3 Classes de données

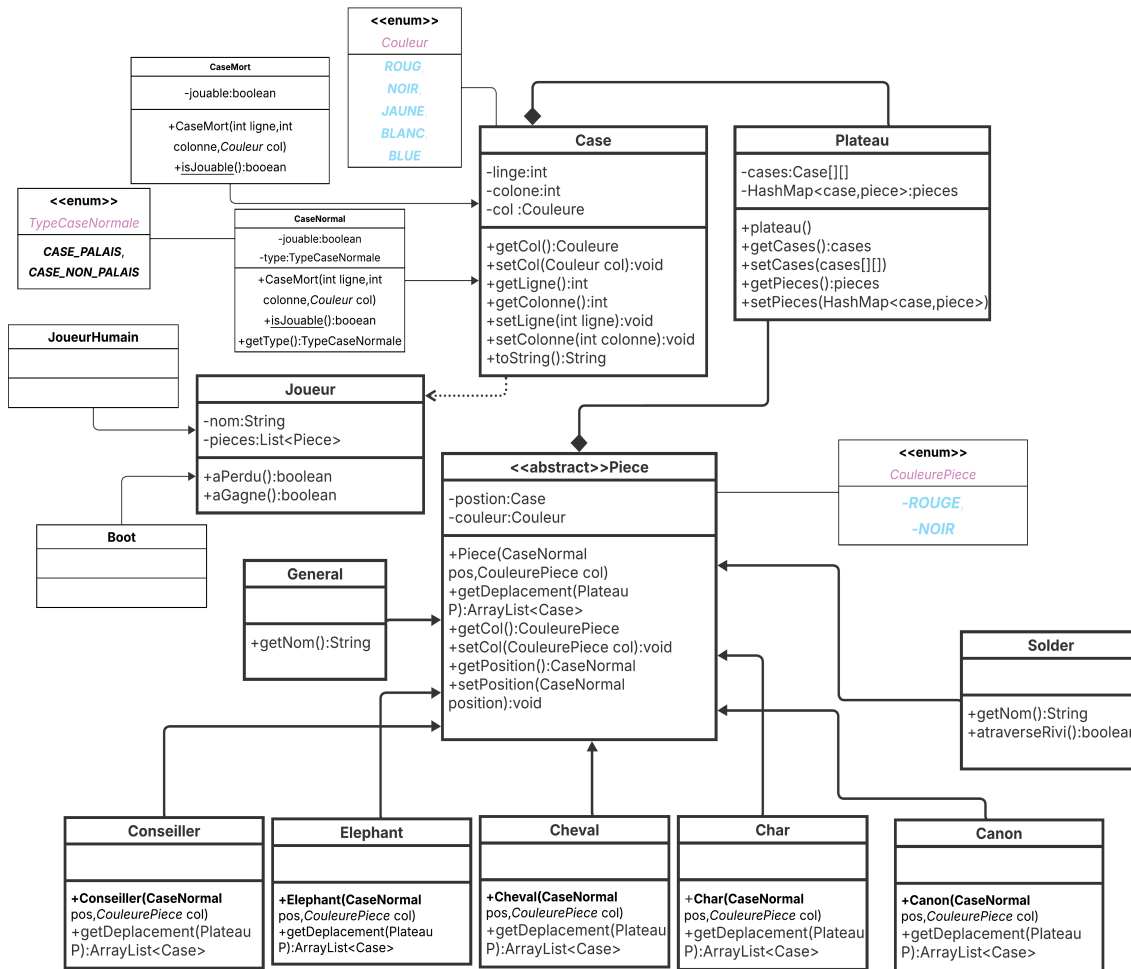


FIGURE 3 – Diagramme de classes

3.3.1 La classe Plateau

La classe Plateau est au cœur du projet. Elle représente l'état complet du jeu à tout moment. Elle contient :

- un tableau `Case[11][11]` qui représente la grille du plateau ;
- une `HashMap<Case, Piece>` qui associe chaque case occupée à sa pièce ;
- la couleur du joueur dont c'est le tour (`CouleurePiece`) ;
- deux listes pour stocker les pièces capturées (une par couleur).

3.3.2 Initialisation du plateau

Au démarrage, les cases neutres (colonnes 3 et 7) et la rivière (ligne 5) sont initialisées comme des `CaseMort`. Les cases du palais sont de type `CASE_PALAIS`.

3.3.3 Méthodes principales

Les méthodes les plus importantes de `Plateau` sont :

- `coupValide(depart, arrivee)` : simule un coup et vérifie que le Général n'est pas en échec et qu'il n'y a pas de face à face après le coup ;
- `estMenacer(case, couleur)` : vérifie si une case est attaquée par une couleur donnée ;

- `EnEchec(joueur)` : dit si un joueur est actuellement en échec ;
- `EnEchecEtMat(joueur)` : vérifie si un joueur n'a plus aucun coup légal ;
- `RoiFacAfac()` : détecte si les deux Généraux sont sur la même colonne sans pièce entre eux.

3.3.4 La hiérarchie des pièces

La classe abstraite `Piece` définit la méthode `getDeplacement(Plateau)` que chaque sous-classe redéfinit selon ses propres règles. Par exemple, le `Canon` parcourt les cases horizontalement et verticalement en ignorant les colonnes neutres, et cherche un écran avant de pouvoir capturer.

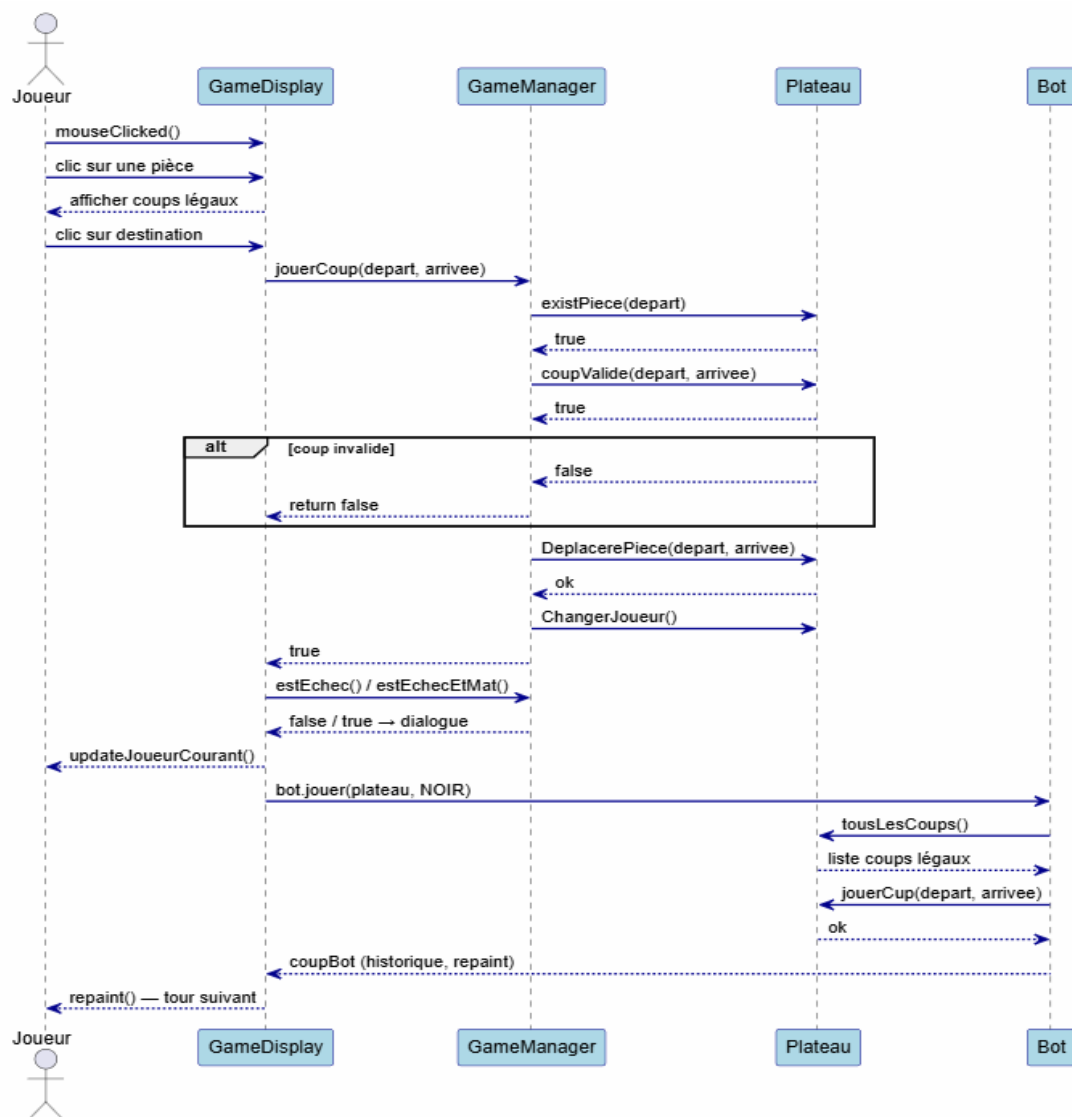
3.4 IHM Graphique

L'interface graphique est faite avec **Java Swing** et est composée de trois classes principales :

- `PageAcc` : la page d'accueil avec un fond d'image, le titre du jeu, et les boutons pour choisir le mode de jeu. En mode VS BOT, un second dialogue apparaît pour choisir la difficulté ;
- `GameDisplay` : le panneau principal pendant la partie. Il contient le plateau au centre, les zones de captures en haut et en bas, et un panneau à droite avec l'historique des coups et le bouton menu ;
- `PaintStrategy` : s'occupe uniquement du rendu graphique — dessine les cases, les pièces (images PNG) et les surbrillances des coups légaux.

3.5 Conception des traitements (processus)

3.5.1 Diagramme de sequence



3.5.2 Les niveaux du bot

Le bot dispose de trois niveaux définis dans l'énumération Niveau :

- **Facile** : le bot choisit un coup au hasard parmi tous ses coups légaux ;
- **Moyen** : le bot essaie de capturer la pièce adverse la plus précieuse, en vérifiant d'abord que la case d'arrivée n'est pas dangereuse pour lui ;
- **Difficile** : le bot calcule un score pour chaque coup possible. Ce score prend en compte la valeur de la pièce capturée, le danger pour ses propres pièces, la mise en échec de l'adversaire et la sécurité de son Général.

Le tableau 2 montre les valeurs utilisées par le bot pour évaluer les pièces.

Pièce	Valeur
Général	100
Chariot	9
Canon	5
Chevalier	4
Éléphant	3
Conseillère	2
Soldat	1

TABLE 2 – Valeurs des pièces utilisées par le bot

3.5.3 Validation des coups

Dans l'algorithme1, on décrit comment un coup est validé avant d'être joué.

Algorithm 1 Validation d'un coup avant de jouer

Require: Départ d , Arrivée a , Plateau P

Ensure: Vrai si le coup est légal, Faux sinon

if $a \notin \text{getDeplacement}(d, P)$ **then**

return Faux

end if

 Simuler : déplacer la pièce de d vers a

if le Général du joueur courant est menacé **then**

return Faux

end if

if les deux Généraux sont face à face **then**

return Faux

end if

 Annuler la simulation

return Vrai

3.5.4 Tests unitaires

Les tests unitaires sont dans la classe `PlateauTest` et utilisent JUnit. On a testé les cas suivants :

- Le plateau et les pièces sont bien initialisés au démarrage ;
- Le premier joueur est bien ROUGE ;
- Le changement de joueur fonctionne correctement après un coup ;
- La détection de présence ou d'absence de pièce sur une case ;
- Un coup valide est bien accepté, un coup invalide est bien refusé ;
- La détection des cases de bordure ;
- Les deux Généraux sont bien présents sur le plateau ;
- Aucun joueur n'est en échec au début de la partie ;
- Une capture est bien enregistrée dans la liste correspondante ;
- Les Généraux ne sont pas face à face en position initiale.

4 Manuel utilisateur

4.1 Prérequis

Pour faire tourner CHESS, il faut :

- Java JDK 1.8 ou supérieur installé sur la machine ;
- Eclipse IDE ;
- La librairie `log4j-1.2.17.jar` fournie par l’enseignant.

4.2 Importer et configurer le projet sous Eclipse

Importer le projet :

1. Ouvrir Eclipse
2. Aller dans `File` → `Import`
3. Choisir `Existing Projects into Workspace`
4. Sélectionner le dossier du projet et valider

Ajouter la librairie Log4j :

1. Clic droit sur le projet → `Properties`
2. Aller dans `Java Build Path` → `Libraries`
3. Cliquer sur `Add External JARs` et sélectionner `log4j-1.2.17.jar`
4. Valider avec `Apply and Close`

Lancer le jeu :

1. Aller dans le package `gui`
2. Faire un clic droit sur `MainGUI.java`
3. Choisir `Run As` → `Java Application`

4.3 Utilisation du jeu

Page d’accueil Au lancement, on arrive sur la page d’accueil avec deux boutons :

- **VS BOT** : jouer contre l’ordinateur. Un menu s’ouvre pour choisir la difficulté (Facile, Moyen ou Difficile) ;
- **VS HUMAIN** : jouer à deux joueurs sur la même machine, chacun son tour.

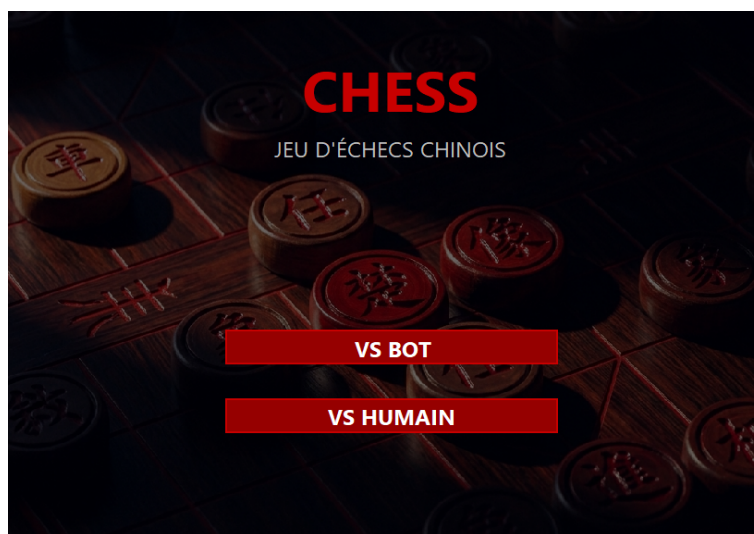


FIGURE 4 – Page d’accueil de CHESS

Pendant une partie

- Pour jouer, on clique d'abord sur une de nos pièces — les cases où elle peut aller s'affichent en surbrillance ;
- On clique ensuite sur une case en surbrillance pour effectuer le déplacement ;
- Si on clique ailleurs, la sélection est annulée ;
- Les pièces capturées apparaissent dans les panneaux en haut (noires capturées) et en bas (rouges capturées) du plateau ;
- L'historique des coups joués s'affiche dans le panneau à droite avec la couleur du joueur correspondant ;
- L'indicateur en bas à droite montre en permanence à qui c'est le tour (RED ou BLACK).

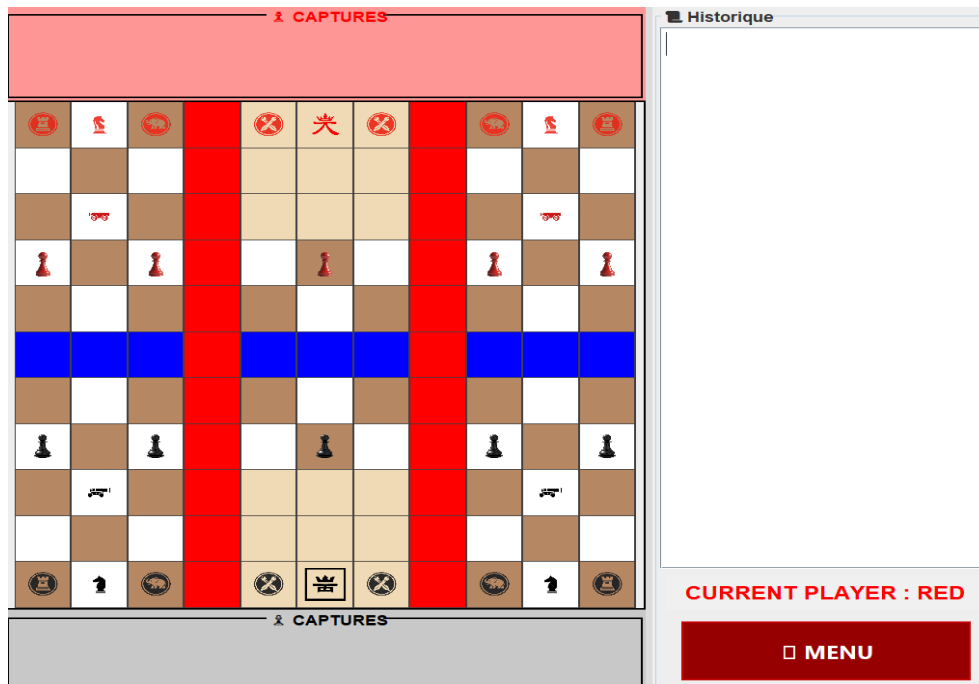


FIGURE 5 – Plateau de jeu pendant une partie

Menu en cours de partie Le bouton **MENU** ouvre un dialogue avec trois options :

- **Reprendre** : ferme le menu et continue la partie ;
- **Nouvelle Partie** : remet le plateau à zéro ;
- **Quitter** : retourne à la page d'accueil.



FIGURE 6 – Menu en cours de partie

Fin de partie Quand un joueur est en échec, une boîte de dialogue l'avertit. Quand il est en échec et mat, la partie se termine automatiquement et on ne peut plus jouer.

5 Déroulement du projet

5.1 Réalisation du projet par étapes

On a découpé le projet en plusieurs étapes successives :

1. Lecture et analyse des spécifications, puis conception de l'architecture MVC et des diagrammes UML ;
2. Mise en place des classes de base : plateau, cases, hiérarchie des pièces ;
3. Implémentation des règles de déplacement de chaque pièce, en gérant les colonnes neutres ;
4. Développement de la logique de validation des coups, de la détection d'échec et de la règle face à face ;
5. Développement de l'interface graphique avec Java Swing ;
6. Implémentation du bot et de ses trois niveaux de difficulté ;
7. Mise en place du système de logs avec Log4j ;
8. Écriture des tests unitaires JUnit ;
9. Rédaction du rapport.

5.2 Répartition des tâches, communication et intégration

KETFI Ramdane S'est occupé du moteur de jeu et des classes de données : le plateau, la hiérarchie des pièces, la validation des coups, la détection d'échec, l'échec et mat, et la règle du face à face.

SOW Mamadou Saliou S'est occupé de l'interface graphique : la page d'accueil, le plateau de jeu, l'affichage des pièces capturées et l'historique des coups.

DJERMOUNE Thanina S'est occupée du bot et de ses trois niveaux de difficulté, ainsi que de la rédaction du rapport.

Communication et intégration Pour communiquer et partager notre travail, on a utilisé un groupe WhatsApp où on s'envoyait régulièrement les fichiers du projet sous forme de zip. On a aussi utilisé GitHub pour centraliser le code et éviter les conflits entre nos versions. Cela nous permettait de toujours avoir la dernière version du code et d'avancer ensemble malgré nos emplois du temps différents.

Difficultés rencontrées Au cours du projet, nous avons fait face à deux types de difficultés :

- **Difficultés organisationnelles** : Des divergences d'opinions ont parfois émergé au sein de l'équipe concernant certains choix techniques et de conception. Cependant, nous avons toujours privilégié le dialogue et la recherche d'un consensus, ce qui nous a permis de maintenir une cohésion solide tout au long du projet.
- **Difficultés techniques** : La modélisation initiale du plateau et la hiérarchie des classes, notamment la gestion des colonnes neutres et de la rivière, ont nécessité une réflexion approfondie sur l'architecture du projet. L'intégration du bot au moteur de jeu a également été complexe, en particulier la conception d'un algorithme d'évaluation cohérent pour les niveaux Moyen et Difficile. Ces défis nous ont poussés à revoir certaines de nos décisions de conception, et nous en sommes sortis avec une meilleure compréhension du développement logiciel en équipe.

6 Conclusion et perspectives

6.1 Résumé du travail réalisé

Au final, on a réussi à développer un jeu d'échecs chinois fonctionnel avec les éléments suivants :

- Un plateau 11×11 avec colonnes neutres et rivière, qui respecte toutes les spécifications ;
- Les règles de déplacement de toutes les pièces, avec la gestion correcte des colonnes neutres ;
- Un moteur de jeu qui valide chaque coup, détecte l'échec, l'échec et mat, et la règle du face à face ;
- Un mode deux joueurs et un mode contre le bot avec trois niveaux de difficulté ;
- Une interface graphique Swing avec historique des coups et affichage des captures ;
- Un système de logs avec Log4j et des tests unitaires avec JUnit.

Ce projet nous a vraiment aidés à comprendre l'importance de bien séparer les responsabilités dans un logiciel, et à travailler en équipe sur un projet Java de taille réelle.

6.2 Améliorations possibles du projet

Si on avait eu plus de temps, on aurait voulu :

- Améliorer le bot avec un algorithme Minimax et l'élagage alpha-bêta pour le rendre plus fort ;
- Ajouter une fonctionnalité de sauvegarde et de chargement de partie ;
- Ajouter la rotation automatique du plateau en mode deux joueurs pour que chaque joueur voie son côté ;
- Ajouter des animations pour les déplacements de pièces ;
- Permettre de jouer en réseau contre un autre joueur.

Références

- [Apa12] Apache. Apache log4j 1.2, 2012.
- [Ora14] Oracle. Java se 8 documentation, 2014.
- [Ora24] Oracle. Java swing tutorial, 2024.
- [Tea24] JUnit Team. Junit 4 documentation, 2024.
- [Wik24] Wikipedia. Xiangqi — Échecs chinois, 2024.